

Documento de Arquitetura de Software

<i>[PJ] – [Plantão já]</i>
Pedro Ribeiro Filho
João Pedro Ferreira
Matheus do Valle
Caio Villela Mouta
Rômulo Ferreira Goes
João Marcos Martins Barbalho da Silva

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

INTRODUÇÃO

1.1 Finalidade

O Plantão Já é uma plataforma web desenvolvida com o objetivo de facilitar a gestão de plantões hospitalares por meio de uma solução digital centralizada.

Durante a etapa inicial do projeto foi realizada a concepção arquitetural da solução, contemplando requisitos funcionais, requisitos não funcionais e uma proposta de arquitetura orientada a serviços.

Na etapa de implementação, parte significativa dessa arquitetura foi concretizada por meio da utilização de microsserviços desenvolvidos em Spring Boot, integração com banco de dados PostgreSQL, autenticação baseada em JWT e uma interface web construída com React.

O sistema atualmente permite o gerenciamento de usuários, hospitais e plantões, fornecendo uma base sólida para futuras evoluções da plataforma.

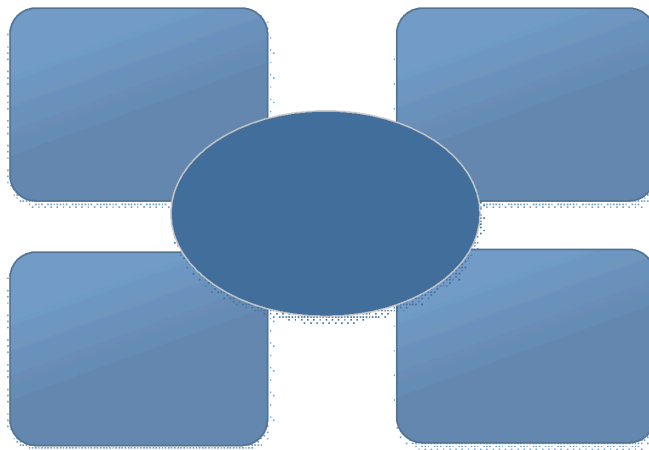


Figura 1 – Arquitetura 4+1

Escopo

O escopo da versão atual do Plantão Já contempla a implementação dos módulos essenciais para operação da plataforma.

Funcionalidades implementadas:

- Autenticação de usuários utilizando JWT.
- Cadastro e gerenciamento de usuários.
- Cadastro e gerenciamento de hospitais.
- Cadastro e gerenciamento de plantões.
- Interface web desenvolvida em React.
- APIs REST desenvolvidas em Spring Boot.
- Persistência de dados utilizando PostgreSQL.
- Arquitetura baseada em microsserviços.

Funcionalidades previstas para evolução futura:

- Sistema de avaliação de profissionais.
- Sistema de candidatura a plantões.
- Integrações com serviços externos.
- Sistema avançado de notificações.

REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento são:

Visão	Público	Área	Modelo da MDS
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Casos de uso
Lógica	Analistas	Realização dos Casos de Uso	Modelo Conceitual, Diagrama de Classes
Implementação	Programadores	Componentes de Software	Diagramas de Atividade, Sequência, Interação e Comunicação
Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação

Tabela 1 – Visões, Público, Área e Artefatos da MDS

1.1 Requisitos e restrições arquiteturais

Esta seção descreve os requisitos de software e restrições que têm um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	JavaScript/TypeScript no frontend; Spring/Java no backend
Plataforma	Docker para containerização; Ngrok/Heroku/AWS/Cloudflare para deploy rápido e escalável
Segurança	Foco em proteção de dados privados de plantonistas/gestores via criptografia e autenticação
Persistência	Sim, usando um banco de dados relacional criptografado para armazenar os dados dos usuários e dos plantões. Para o ingresso dos plantões, um cache em redis para dados transientes(para verificar quem clicou primeiro no caso de dois plantonistas ingressarem ao mesmo tempo)

Internacionalização (i18n)	Não necessário inicialmente, priorizando PT-BR para público local
Modularização	Sim, microserviços isolado para ingressos de plantonistas, para lidar com picos ao lançar plantões novos, via queues ou load balancing
Performance	Tempo de resposta <2s para ingressos, o que também justifica o uso do redis.
Coexistência	O sistema deve fazer scrapings em outros serviços/websites, porém, caso não haja disponibilidade do serviço alvo, a consulta deve ficar em uma fila de espera, não travando o processo inicial do sistema. Para isso, haverá um módulo de consultas externas independente do sistema original.
Disponibilidade	99,9% uptime com health checks, usando redundância em cloud

Tabela 2 – requisitos e restrições

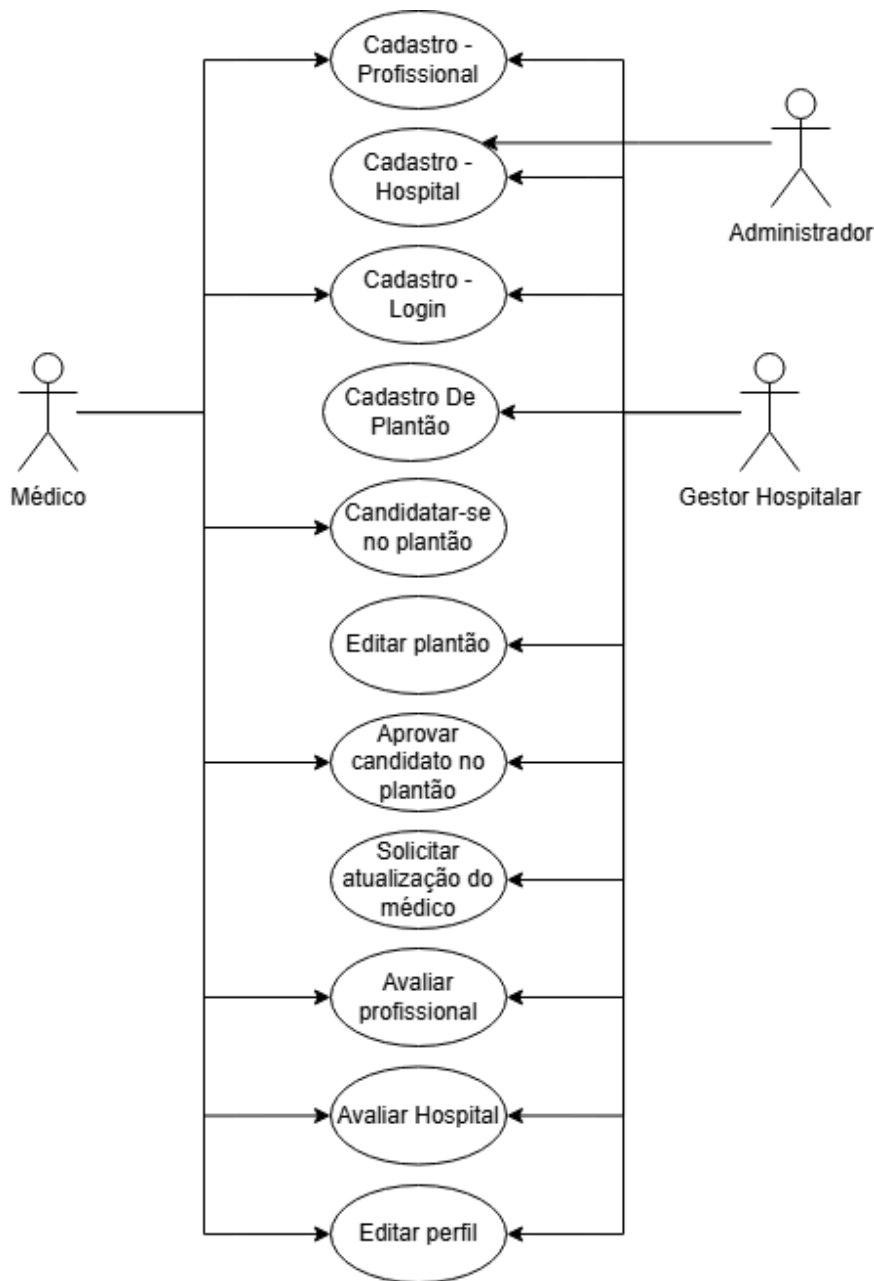
VISÃO DE CASOS DE USO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.

Lista de casos de uso do sistema:

- Cadastro - Profissional(Gestor e Médico) [000]
- Cadastro - Hospital [001]
- Cadastro - Login [009]
- Operacional - Cadastrar/Editar plantão [002]
- Operacional - Candidatar-se no plantão [003]
- Transacional - Deletar um plantão [005]
- Operacional - Solicitar atualização do médico(i.e. scraper do jusbrasil e de BO) [006]
- Operacional - Avaliar profissional [007]
- Operacional - Buscar plantão [011]
- Operacional - Editar perfil [008]

Diagrama de casos de uso:

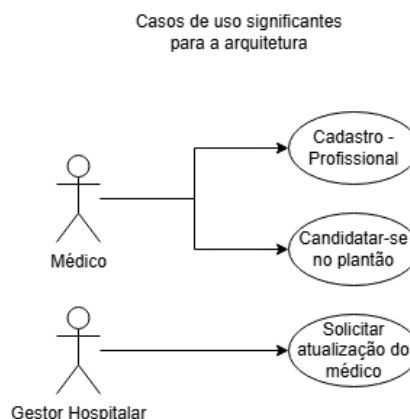


1.2 Casos de Uso significantes para a arquitetura

[000] - Cadastro - Profissional (Privacidade e Conformidade): Embora seja um CRUD, ele lida diretamente com o armazenamento de dados sensíveis (CRM, dados pessoais). Sua significância arquitetural vem da obrigatoriedade de conformidade com a LGPD (RNF6). A arquitetura e o banco de dados relacional precisam prever o isolamento e a criptografia profunda desses dados de saúde e registros profissionais, garantindo que não ocorram vazamentos ou acessos indevidos.

[003] - Ingresso no plantão : Quando a vaga exige aprovação manual do gestor, a ordem temporal das candidaturas não define o candidato selecionado, pois a decisão ocorre com base na análise do perfil profissional. Nesse caso, o timestamp da candidatura pode ser mantido apenas para auditoria, rastreabilidade ou ordenação auxiliar. Já em vagas de ingresso automático, o sistema utiliza serialização de candidaturas concorrentes para garantir consistência na ocupação da vaga em cenários de acesso simultâneo. Para isso, usaremos um banco de dados serializado e monothread para controlar essa ordenação

[006] - Atualização da ficha do médico(envolve acesso em serviços de terceiros, então precisamos contornar possíveis bloqueios de ip e instabilidades):



VISÃO LÓGICA

A Visão Lógica do sistema Plantão já descreve a organização funcional do software, focando em como os requisitos são realizados através de uma estrutura modular de classes e pacotes. A arquitetura segue um modelo de camadas bem definido para garantir o baixo acoplamento e a manutenibilidade: Apresentação (Frontend e Controllers REST), Aplicação (Serviços de negócio e Orquestração), Domínio (Entidades centrais e Regras de Negócio) e Infraestrutura (Persistência e Integrações). As classes mais significativas da arquitetura incluem Profissional (com especializações para Médico e Gestor), Hospital, Plantão e Candidatura, que compõem o núcleo do modelo de domínio. Esses componentes são organizados em subsistemas específicos, como o pacote de Serviços de Candidatura, que isola a lógica de alta concorrência e integração com o Redis, e o pacote de Serviços Externos, responsável pelo scraping assíncrono de dados.

Os aspectos dinâmicos da arquitetura são evidenciados pelas realizações dos casos de uso críticos. Por exemplo, no caso de uso **[003] Candidatar-se no Plantão**, a arquitetura demonstra sua robustez através da interação entre o controlador de candidatura, o serviço de validação e o subsistema de cache monothread para serialização de acessos simultâneos. Da mesma forma, a realização do caso de uso **[006] Atualização da Ficha do Médico** ilustra o comportamento assíncrono do sistema, onde a lógica de negócio delega tarefas para o módulo de scraping via filas de mensagens, protegendo o núcleo do sistema de instabilidades externas. Os relacionamentos detalhados entre esses componentes, pacotes e camadas, bem como o fluxo temporal das mensagens, são ilustrados nos Diagramas de Classes e de Sequência incluídos nesta seção.

1.3 Visão Geral da arquitetura do sistema – pacotes e camadas

Por se tratar de um software que lida com integração com plataformas externas e picos de uso, estruturamos a arquitetura do sistema usando uma combinação de camadas(baseado no MVC), e microserviços. Com isso, conseguimos nos blindar ao

máximo de problemas previstos na fase de concepção do projeto, problemas esses que são:

- Pico de uso: Por se tratar de uma plataforma de plantões, especialmente da área da saúde, temos uma fortíssima concorrência entre os candidatos, o que já tem efeitos latentes na forma como esses plantões são ofertados nos dias de hoje, efeitos esses que são caracterizados por congestionar os meios de comunicação utilizados, que nesse caso será o Plantão Já.

Para solucionar isso, pensamos em utilizar um microserviço direcionado ao ingresso de profissionais nos plantões, permitindo a livre navegabilidade dos outros usuários quando se trata do resto dos serviços, reduzindo o acoplamento.

Ainda pensando nisso, criamos uma integração com um banco de dados Redis, para que pudéssemos controlar a concorrência das inscrições nos plantões, trazendo confiabilidade e segurança ao usuário, garantindo que, caso ele clique primeiro em “Inscrever-se”, tenha de fato sua vaga garantida nos casos em que o gestor opta por não filtrar os profissionais manualmente.

- Serviços externos: Utilizaremos diversos serviços externos para compor as informações de nossos profissionais e estabelecimentos. Por exemplo, ao atualizar a ficha de um médico, precisamos fazer scraping de dados como histórico criminal, processos em aberto, o que pode causar gargalos na atualização geral do perfil. Então, quando um usuário solicita atualização do perfil, todo o processo funciona em um microserviço de “fila”, onde aguarda sua vez de acontecer, sem travar o processo principal.
- Falando sobre cliente-servidor e camadas(MVC), os escolhemos pois esse estilo arquitetural oferece uma separação clara de responsabilidades, o que é essencial para um sistema que precisa equilibrar alta concorrência, integração com serviços externos e evolução contínua. A divisão em camadas - Apresentação (cliente), Controlador (API), serviço (lógica de negócio) e persistência - mapeia naturalmente o modelo MVC e traz os seguintes benefícios para o projeto:
- Manutenibilidade e testabilidade: como a lógica de validação de candidatura fica isolada na camada de serviços, podemos testá-la unitariamente sem depender da interface gráfica ou do banco de dados.

- Independência entre frontend e backend: o frontend se comunica com o backend exclusivamente por meio de APIs REST. Isso permite que equipes diferentes evoluam cliente e servidor em paralelo.
- Adequação ao modelo de microsserviços já definido: cada microsserviço pode ser implementado como uma aplicação MVC própria, expondo seus próprios controladores. A camada de apresentação consolida essas APIs e as apresenta ao usuário final
- Segurança e conformidade; a separação entre camada de apresentação e camada de serviços reduz o risco de exposição acidental de CPFs, CRM ou avaliações.
- Tratamento de serviços externos (scraping) sem bloqueio: quando um médico solicita atualização de sua ficha, o controlador dispara um evento na fila e retorna imediatamente “solicitação recebida”. O processamento pesado ocorre em workers isolados, respeitando o modelo cliente servidor - o cliente não fica aguardando e o servidor principal não trava.

Dessa forma, a escolha pelo modelo cliente-servidor com camadas MVC não é uma contradição com o uso de microsserviços, pois ela organiza a comunicação entre eles e garante que cada subsistema mantenha seu baixo acoplamento e alta coesão.

1.4 Visão Geral – Modelo Conceitual

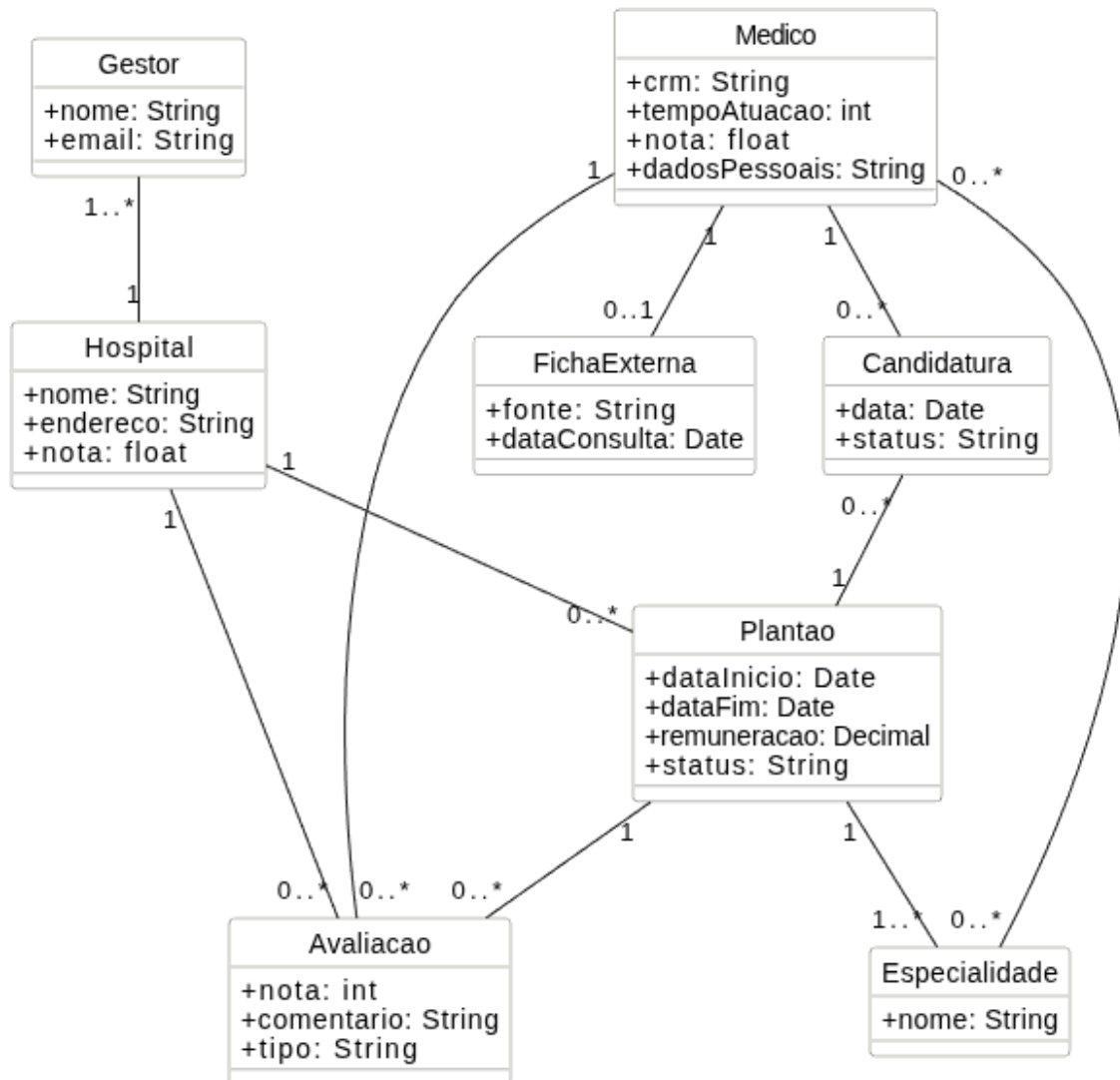


Figura 3 – Diagrama Modelo Conceitual

VISÃO DE IMPLEMENTAÇÃO

5.1 Diagrama de Classes

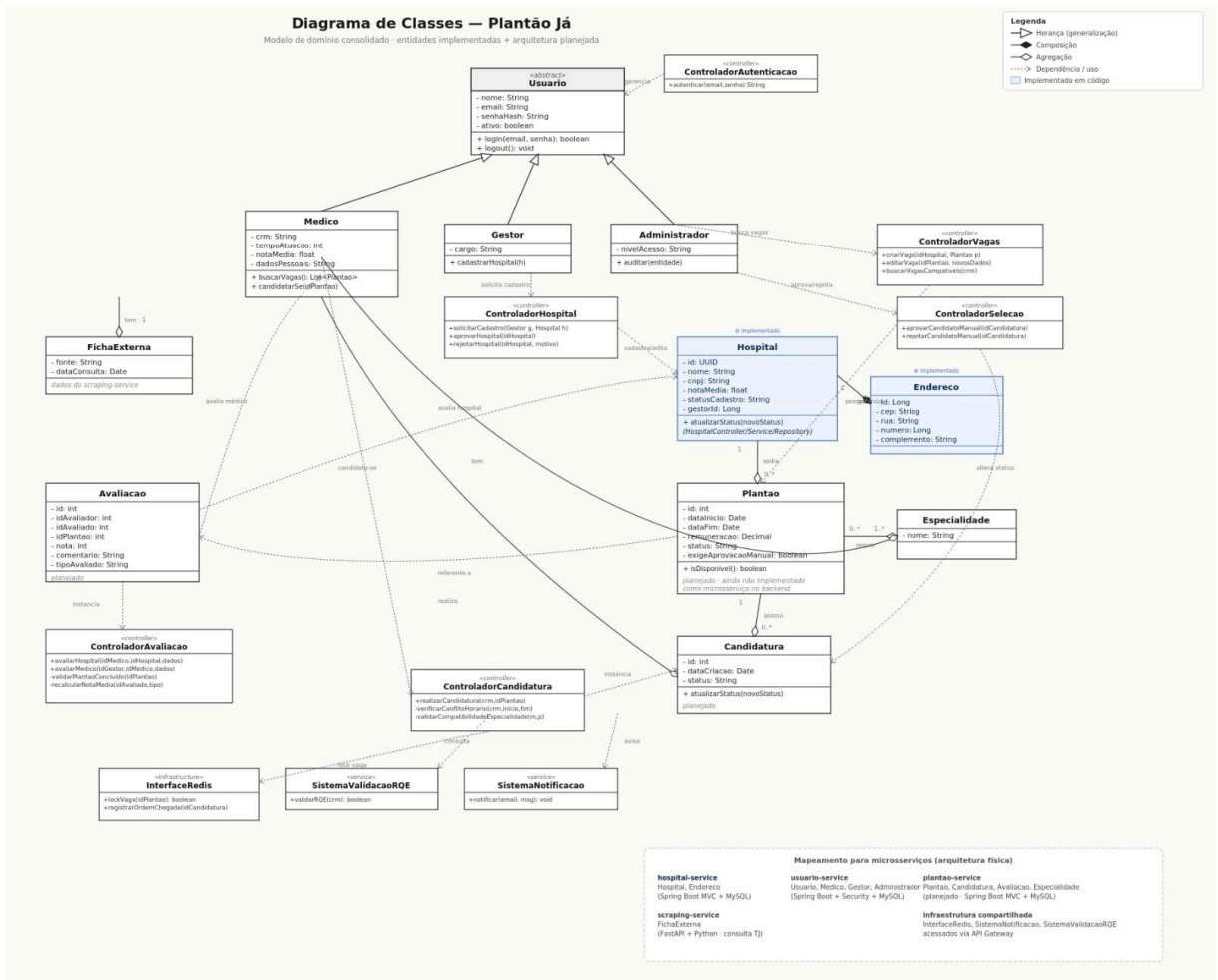
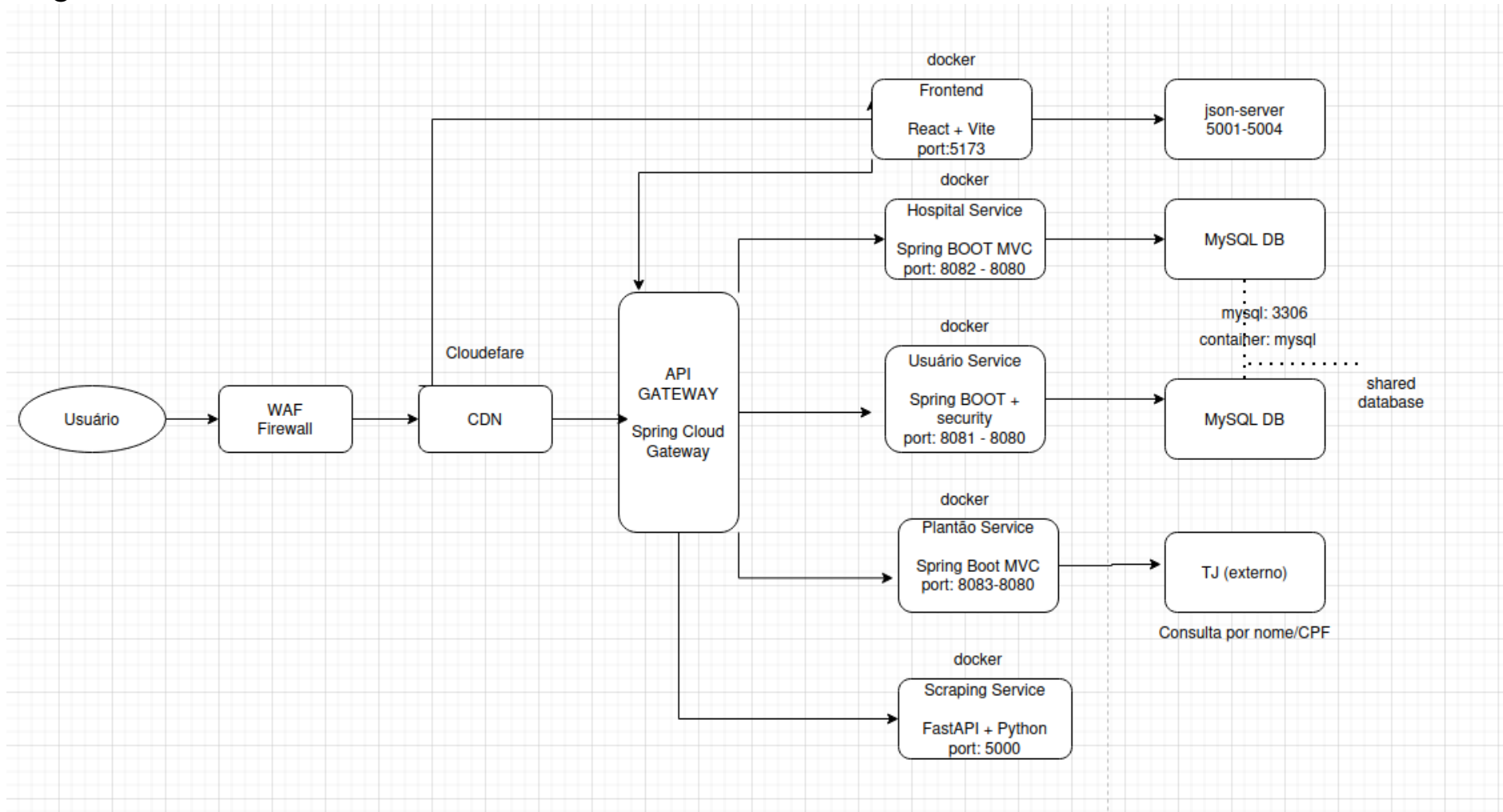


Diagrama estrutural

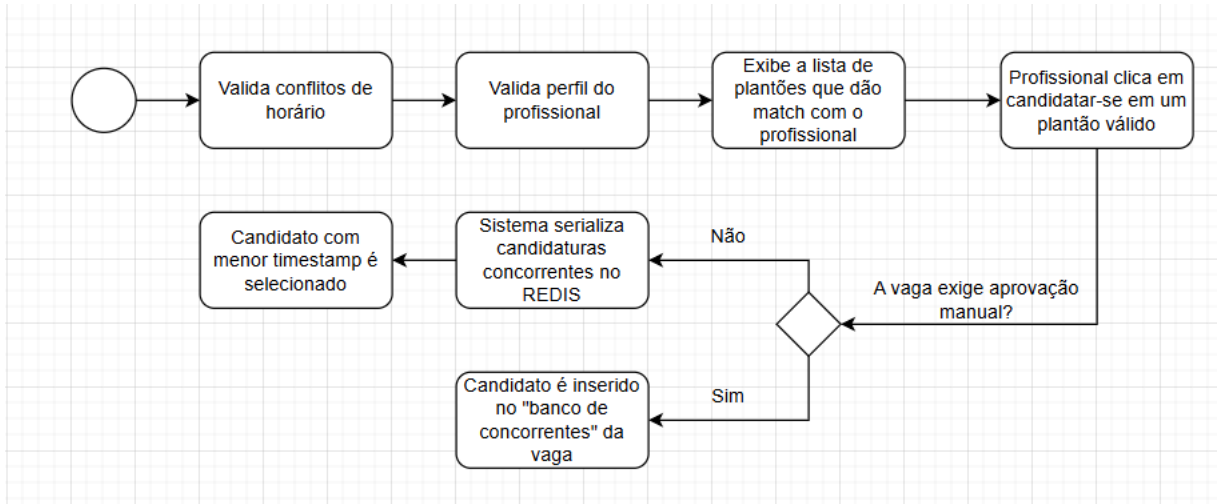


Decisões arquiteturais

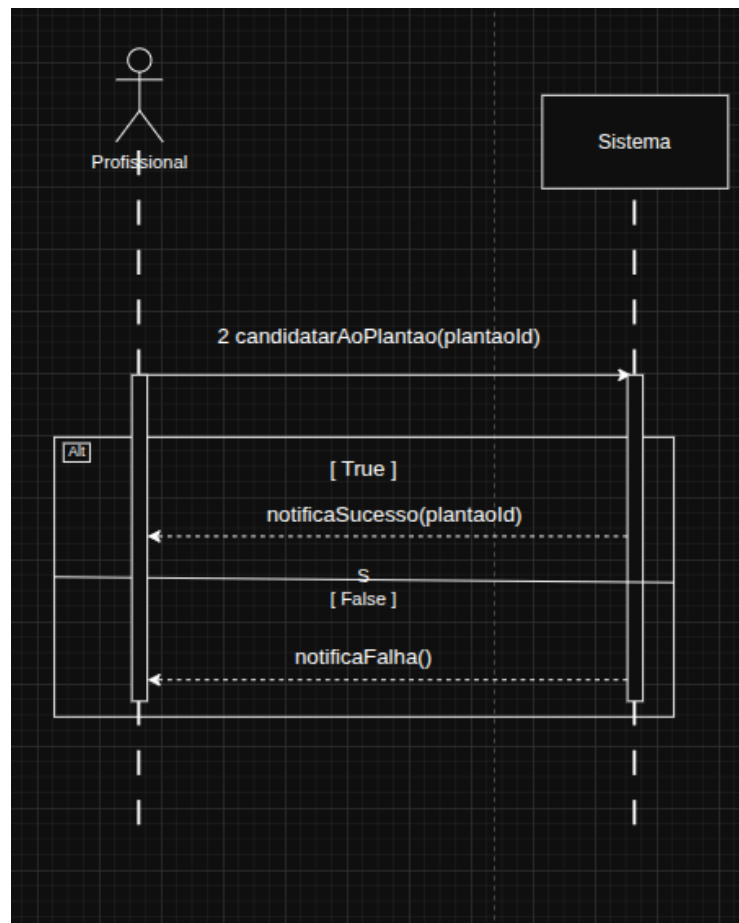
- Foi adotada uma arquitetura baseada em microserviços.
- A solução foi dividida em serviços independentes para usuários, hospitais e plantões.
- Um gateway centraliza o acesso aos serviços.
- A comunicação é realizada através de APIs REST.

5.2 Caso de Uso : Operacional - Candidatar-se no plantão [003]

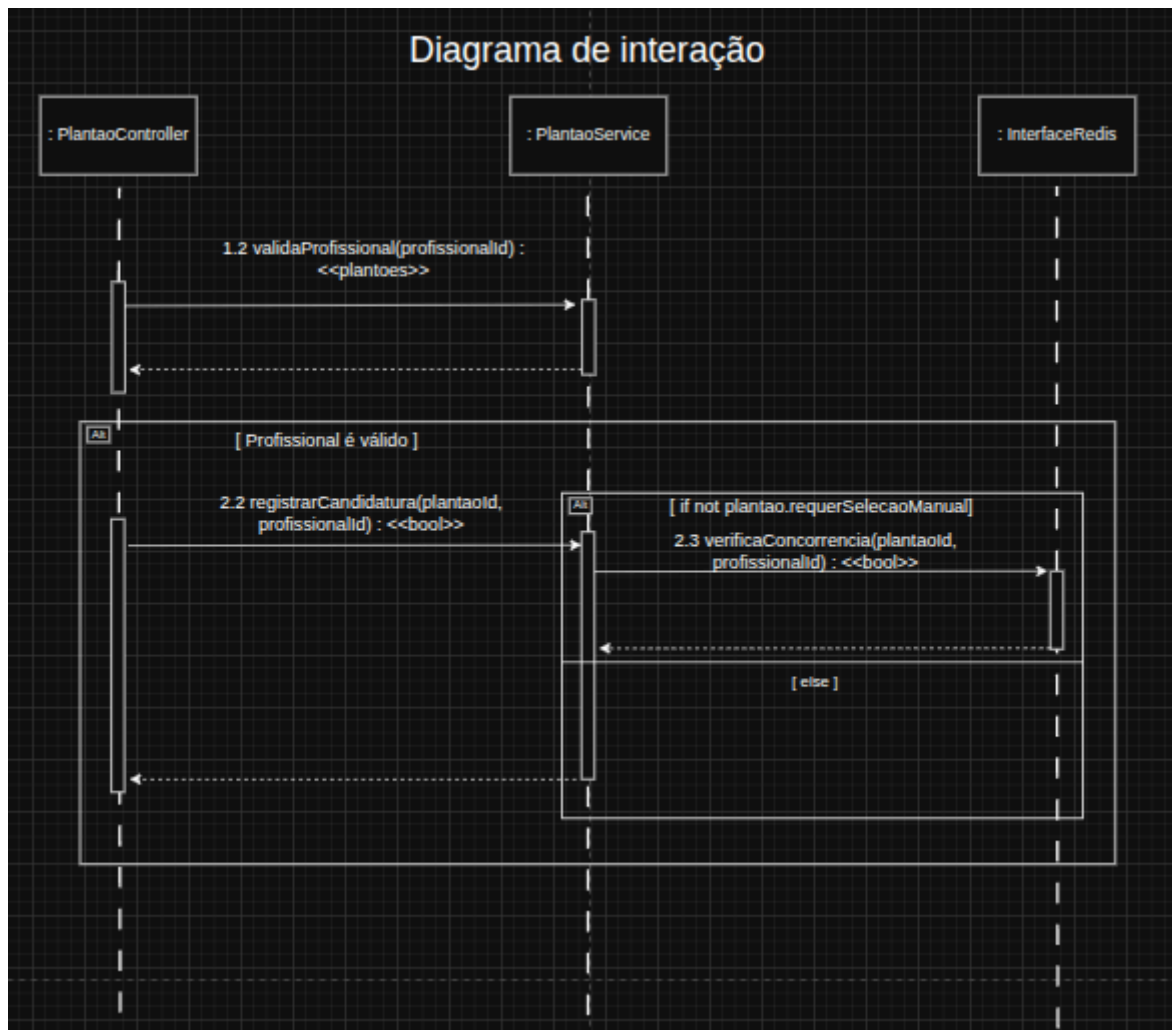
5.2.1 Diagrama de Atividades



5.2.2 Diagrama de Sequência



5.2.3 Diagrama de Interação



5.2.4 Descrição da operação **registrarCandidatura(plantaoid, profissionalId)** :

Responsabilidade:

Realizar, sistematicamente, o registro do profissional no referido plantão. A operação valida a elegibilidade do profissional, tanto profissionalmente quanto no quesito da disponibilidade de horários. Quando for o caso de não haver seleção manual dos inscritos, trabalha com regras de concorrência temporal, utilizando um banco de dados serializado para lidar com possíveis candidaturas simultâneas entre profissionais. Já quando for o caso da vaga ser supervisionada, o salvamento é feito no “banco de talentos” referente à vaga.

Observação arquitetural: a validação da elegibilidade do usuário é executada dentro dessa operação, que ocorre dentro de um microserviço, porque por mais que o menu de oportunidades mostre ao profissional apenas as vagas compatíveis, ele ainda pode acessar a vaga por outros locais, seja no menu geral de vagas, ou diretamente pelo link. Com isso em mente, deixar

essa responsabilidade de validação para a view ou para o controller gera um acoplamento ruim entre a navegação e a regra de negócio, além de gerar overhead no servidor principal. Realizando a validação diretamente no microserviço de plantões, tiramos esse peso do servidor principal, além de tirarmos a validade do contrato da pré-condição, deixando a operação mais autocontida.

Pré condições:

- Plantão e funcionário existem
- Plantão é válido e está com candidatura aberta
- Funcionário é válido

Pós condições de sucesso:

- O funcionário está cadastrado no plantão

Pós condições de falha:

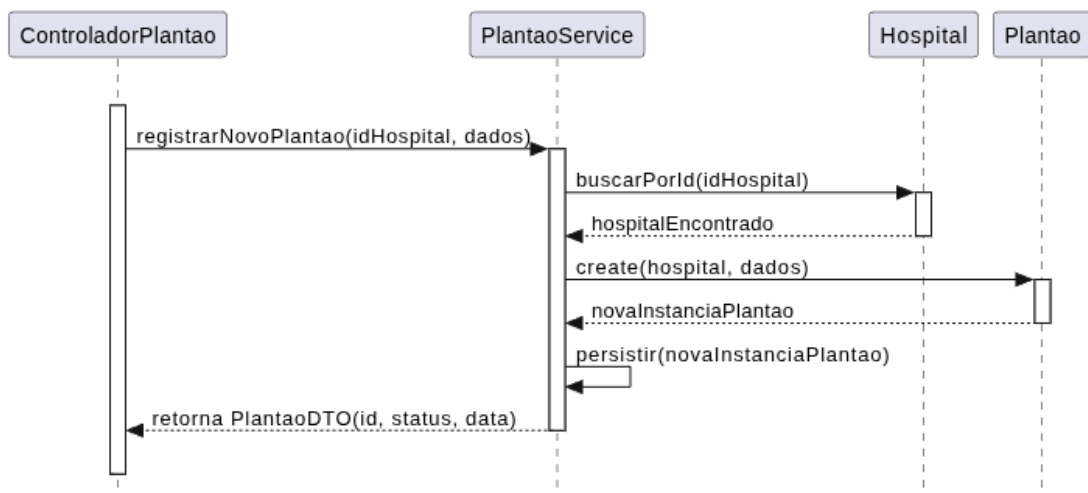
- O funcionário é notificado que houve falha

5.3 Caso de Uso : Operacional - Cadastrar plantão [002]:

5.3.1 Diagrama de Sequência



5.3.2: Diagrama de interação



5.3.3 Descrição das operações **registrarNovoPlantao(idHospital, dados):**

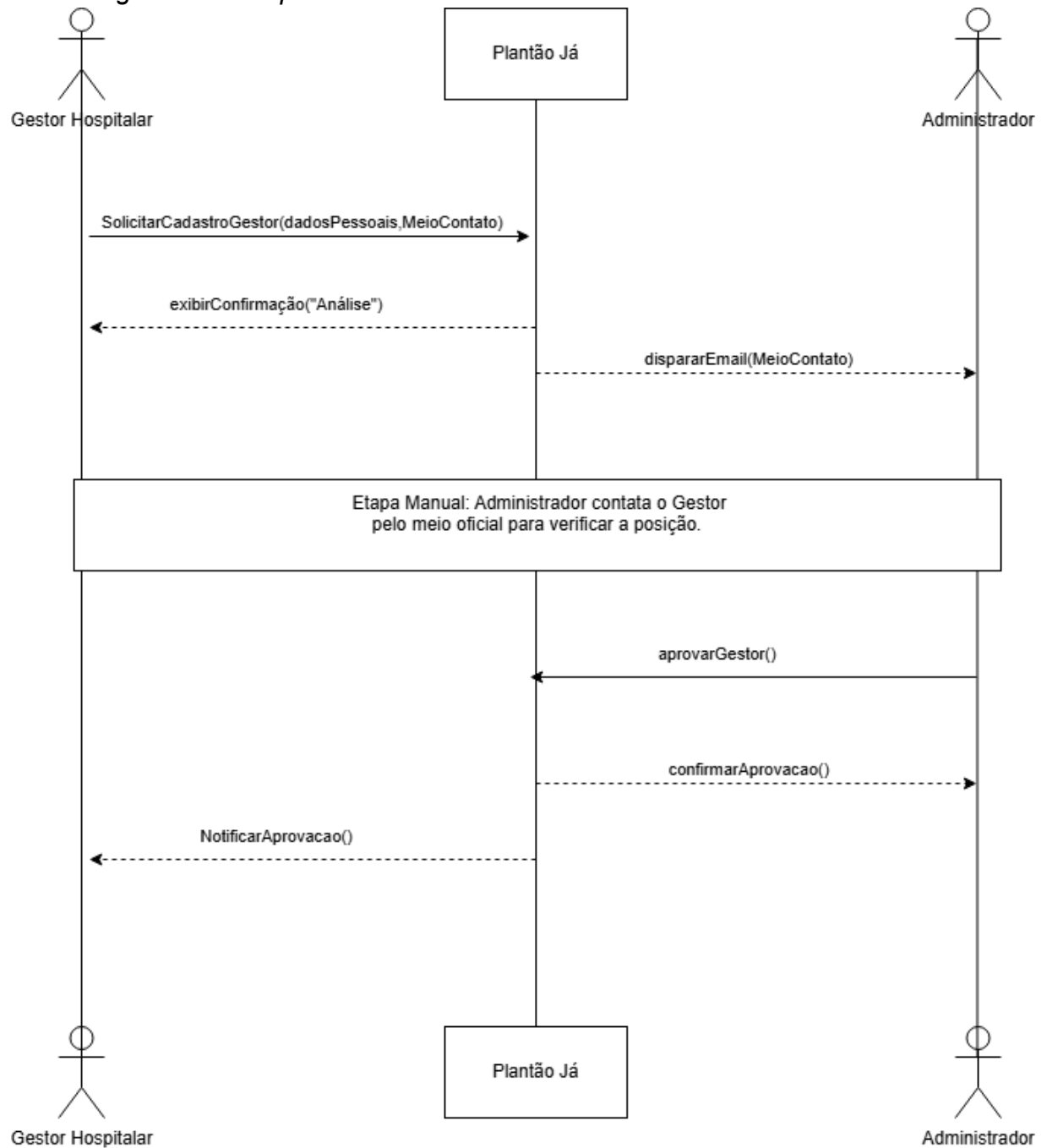
Responsabilidade: Realizar o registro de uma nova oferta de plantão vinculada a um Hospital. A operação **criarPlantao** recebe os dados da vaga, valida o estado do Hospital agregado e persiste o Plantão com status inicial “aberto”, disparando um evento de notificação para os médicos elegíveis.

Pré-condições:

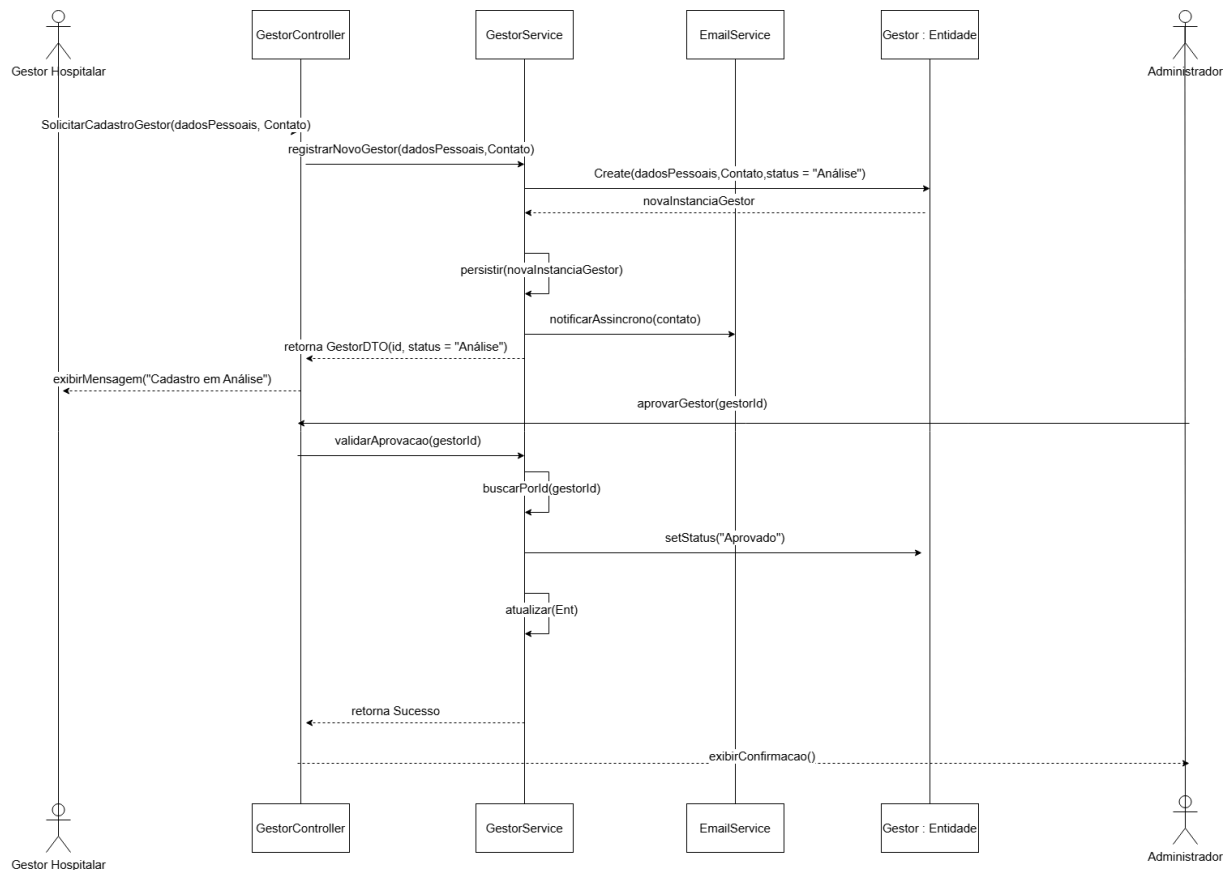
- O Gestor deve estar autenticado e vinculado ao Hospital no sistema;
- O Hospital referenciado por idHospital deve existir e estar em estado ativo na plataforma;
- Os dados do plantão devem conter o hospital do plantão, data de início, data de término, especialização exigida e valor da hora preenchidos e válidos.

5.4 Caso de Uso : Cadastro - Hospital [001]:

5.4.1 Diagrama de Sequência:



5.4.2: Diagrama de interação:

5.4.3 Descrição das operações **solicitarCadastroGestor(dados, meioContato)** e **aprovarGestor(gestorId)**:

Responsabilidade:

Realizar de forma segura o registro preliminar de um gestor hospitalar e coordenar a liberação de seus acessos. A operação **solicitarCadastroGestor** recebe os dados e cria a entidade em um estado de análise, disparando em seguida um alerta para a administração da plataforma. A operação subsequente, **aprovarGestor**, é executada estritamente pelo Administrador após verificação manual (fora do Sistema), alterando o estado do cadastro para "Aprovado" e concedendo as permissões de criação de plantões.

Observação arquitetural:

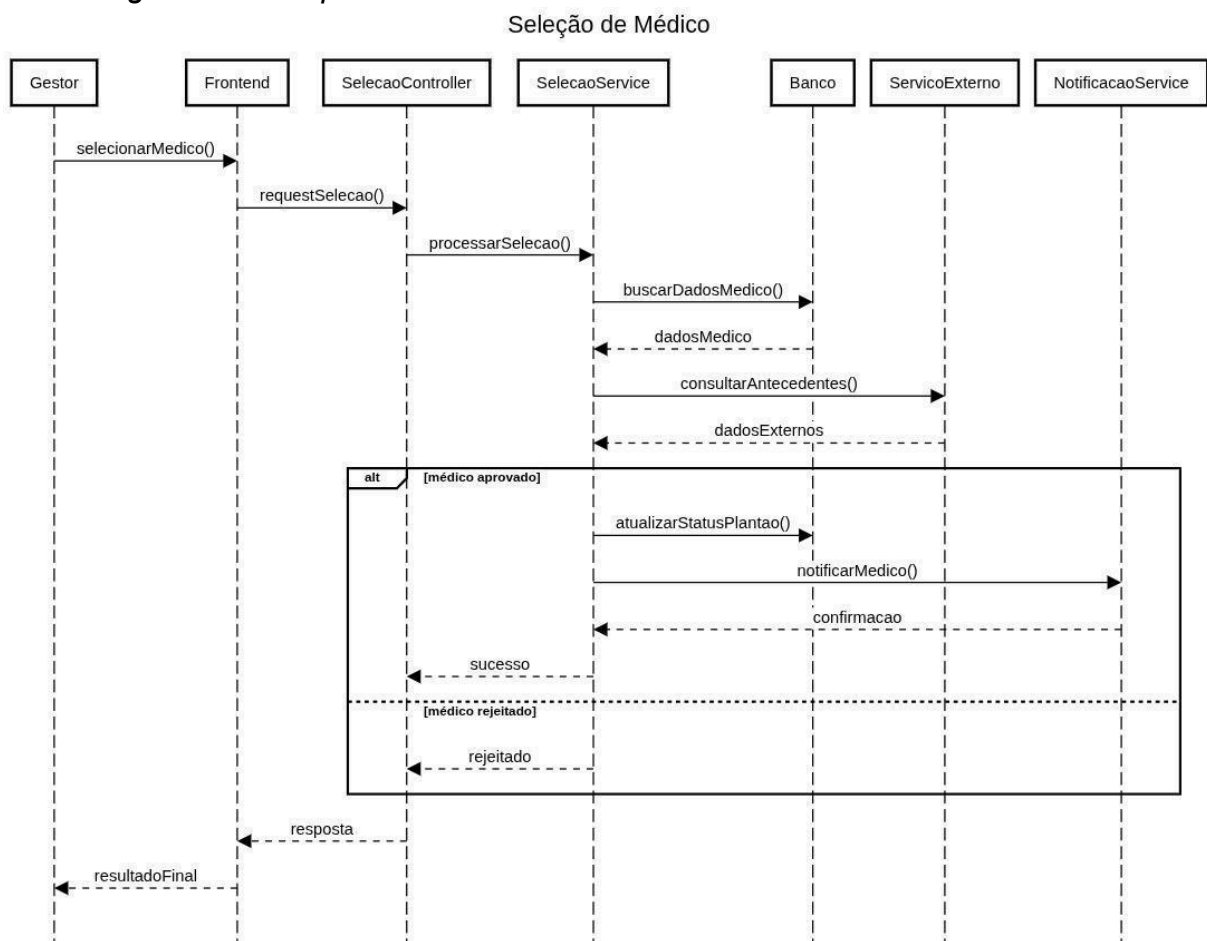
O disparo do e-mail de notificação para o Administrador não deve bloquear a resposta ao usuário. Arquiteturalmente, o ideal é que a criação do Gestor no banco de dados relacional publique um evento em uma fila de mensagens (Mensageria/Assincronismo), e um *worker* dispare o e-mail. Isso evita que uma lentidão no servidor de e-mails cause um *timeout* na tela de cadastro do Gestor. Além disso, a separação de estados ("Em Análise" vs "Aprovado") delega o controle de acesso para a camada de segurança, garantindo que um usuário não validado não consiga realizar bypass para a rota de criação de plantões.

Pré condições:

- Os dados fornecidos pelo Gestor (CPF, e-mail) não podem estar previamente cadastrados no sistema.
- O meio de contato oficial fornecido deve ser válido.
- O Administrador deve possuir privilégios de alto nível (Role: ADMIN) para executar a operação de aprovação

5.5 Caso de Uso : Seleção de médico - Hospital

5.5.1 Diagrama de Sequência:



5.5.2 Descrição das operações atualizarStatusPlantao():

Responsabilidade:

Realizar a atualização do status de um plantão com base no resultado do processo de seleção de médico. A operação `atualizarStatusPlantao()` é responsável por alterar o estado do plantão no sistema, refletindo se houve sucesso na alocação do médico (aprovado) ou não (rejeitado). Essa atualização garante consistência entre o processo de seleção e a disponibilidade real do plantão.

Observação arquitetural:

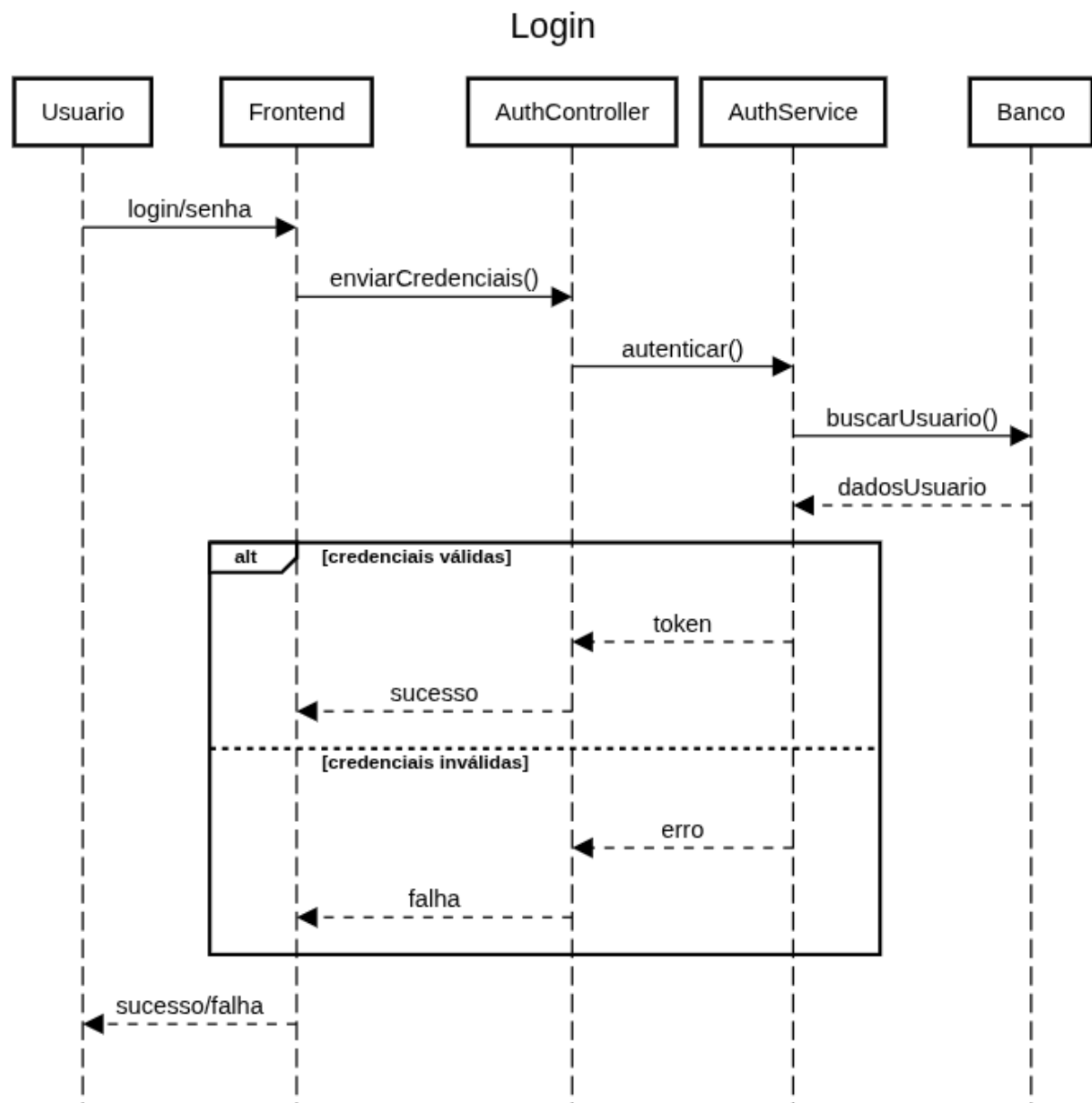
A atualização do status do plantão deve ser uma operação transacional, garantindo que qualquer falha posterior (como erro na notificação) não comprometa a integridade dos dados.

Pré-condições:

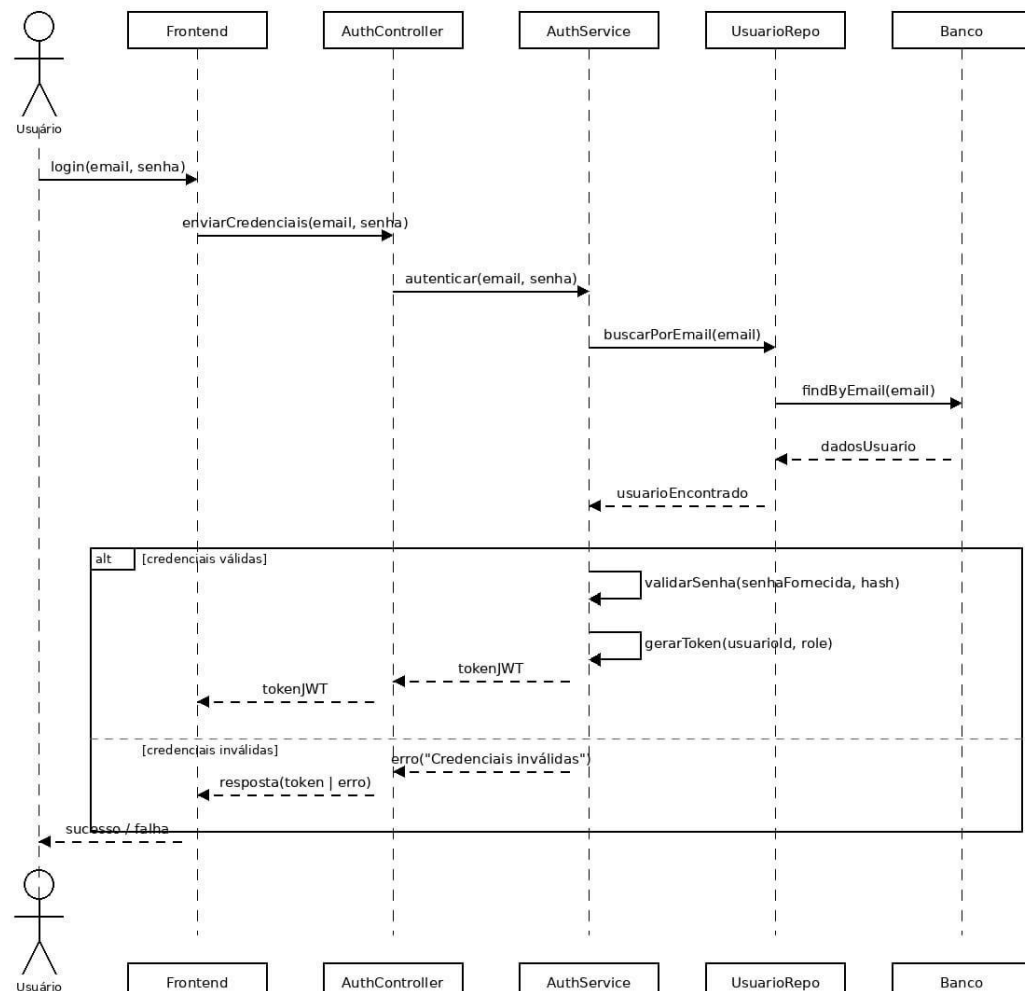
- O plantão deve existir no banco de dados.
- O processo de seleção deve ter sido concluído (com decisão de aprovação ou rejeição).
- O médico avaliado deve estar vinculado ao processo de seleção em questão.

5.6 Caso de Uso : Login:

5.6.1 Diagrama de Sequência



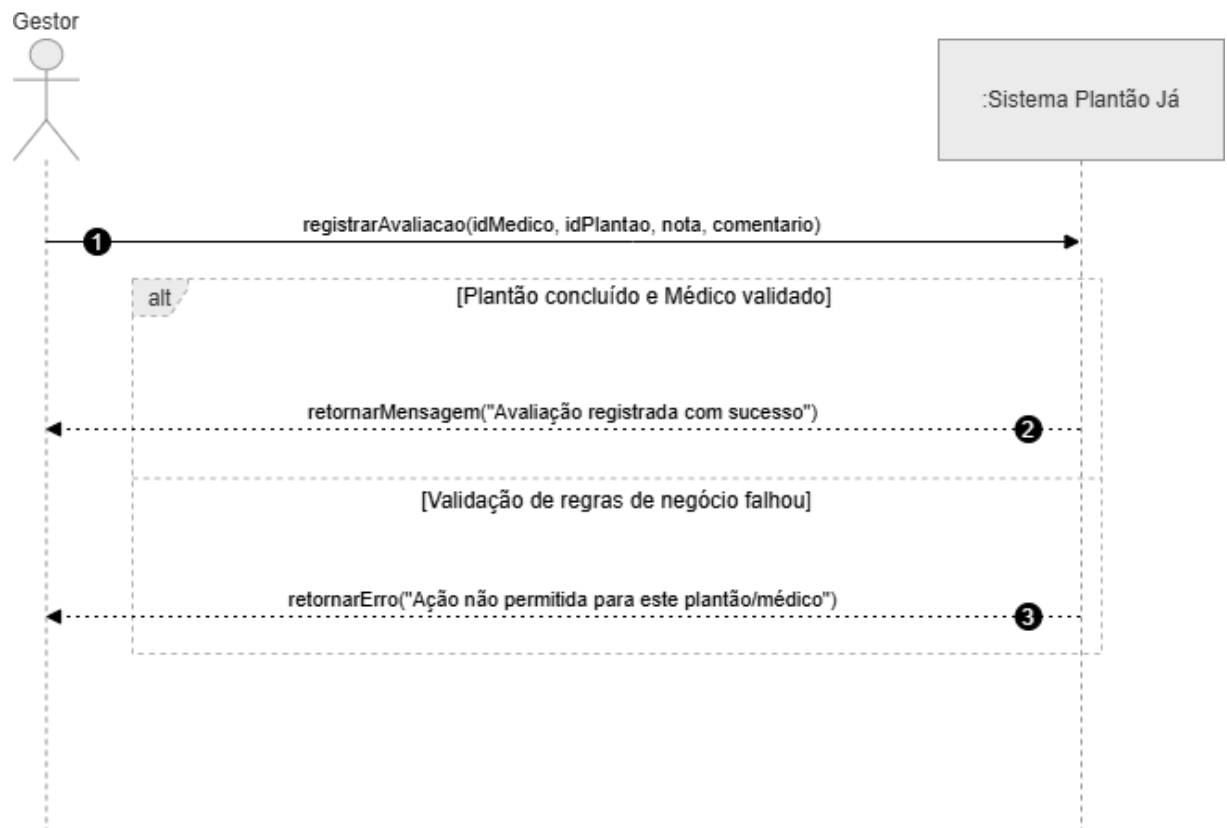
5.6.2 Diagrama de interação:

5.6.3: Descrição da operação:
autenticarUsuario(email, senha):

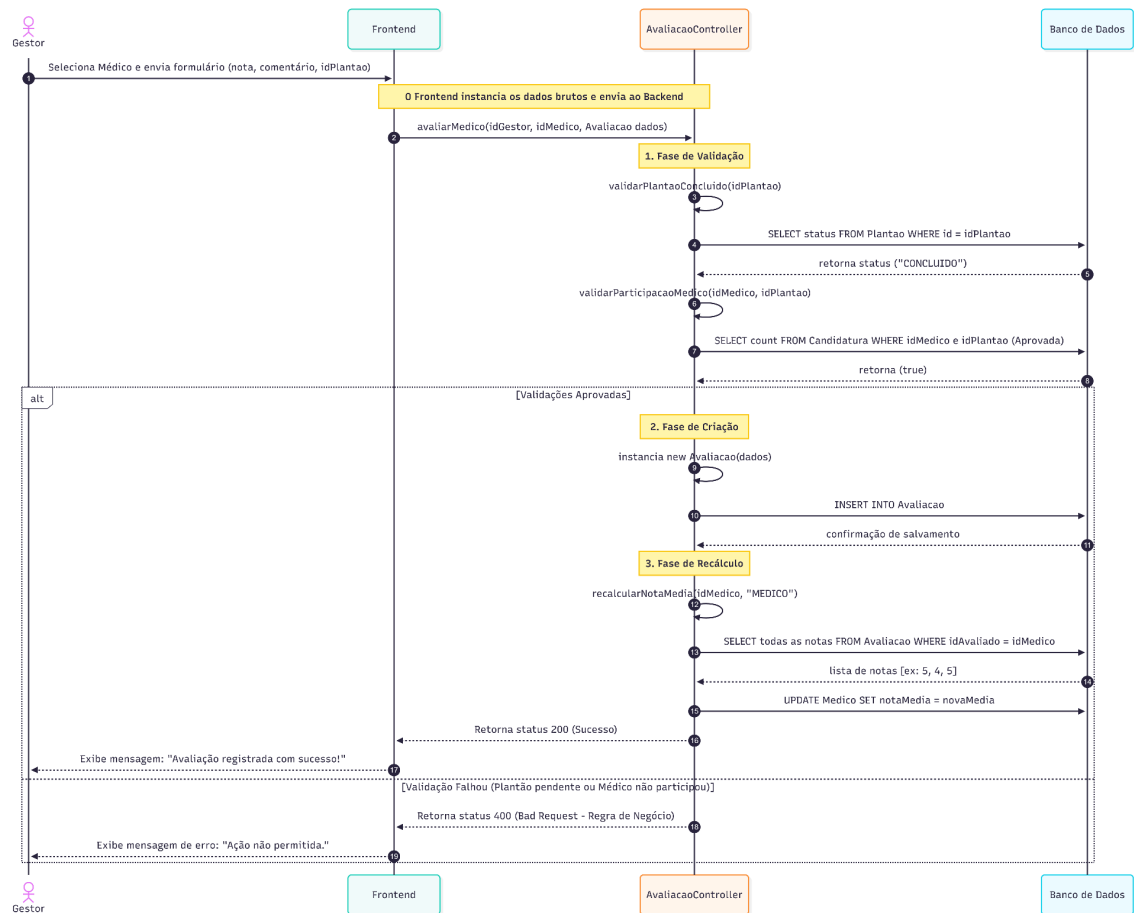
Responsabilidades: verificar as credenciais fornecidas pelo usuário e, em caso de sucesso, estabelecer uma sessão autenticada por meio da emissão de um token JWT, a operação recebe o email e a senha, valida a existência do usuário no sistema, compara o hash da senha armazenada com a senha fornecida e, caso válido, retorna um token de acesso assinado com as informações de perfil encapsuladas. Em caso de falha, a operação retorna uma mensagem genérica de credenciais inválidas, sem especificar qual campo está incorreto, por motivos de segurança.

5.7 - Caso de Uso: Avaliar Médico Plantonista [007]

5.7.1 - Diagrama de Sequência:

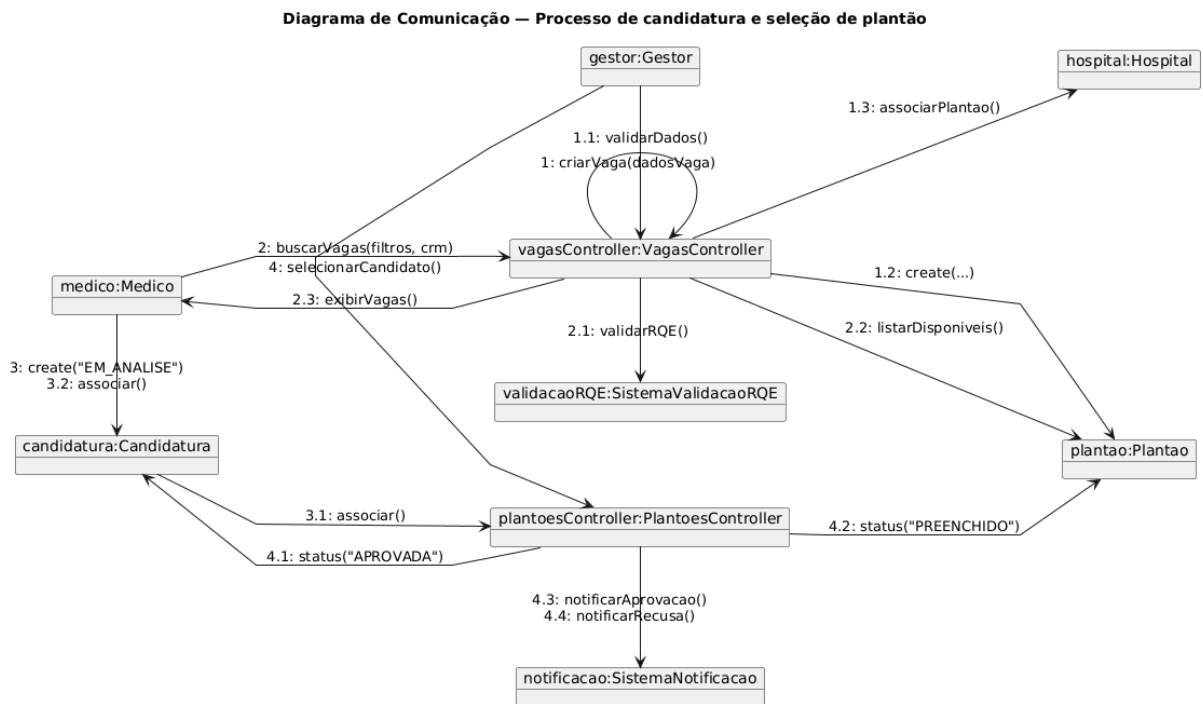


5.7.2 - Diagrama de Interação:

5.7.3 - Descrição da Operação *avaliarMedico(idGestor, idMedico, dadosAvaliacao)*:

Responsabilidades: orquestrar o processo de registro de *feedback* de um gestor sobre um médico plantonista. A operação recebe os identificadores do avaliador (gestor), do avaliado (médico) e o objeto encapsulado contendo os detalhes da avaliação (ID do plantão, nota e comentário). Primeiro, a operação delega validações de segurança e regras de negócios: verifica se o plantão referenciado possui o status “CONCLUÍDO” e confirma se o médico de fato teve sua candidatura aprovada para aquele plantão específico. Em caso de sucesso nas validações, a operação instancia e persiste o objeto **Avaliação** no banco de dados e, logo em seguida, aciona a rotina interna para atualizar a nota média do médico. Em caso de falha nas validações (ex: tentar avaliar um plantão futuro ou um médico não participante), a operação é abortada e retorna uma mensagem de erro de regra de negócio, impedindo a gravação de avaliações fraudulentas.

5.8 - Diagrama de comunicação - Candidatura e Seleção de Candidatos



VISÃO DE IMPLANTAÇÃO

1.5 Diagrama geral da arquitetura

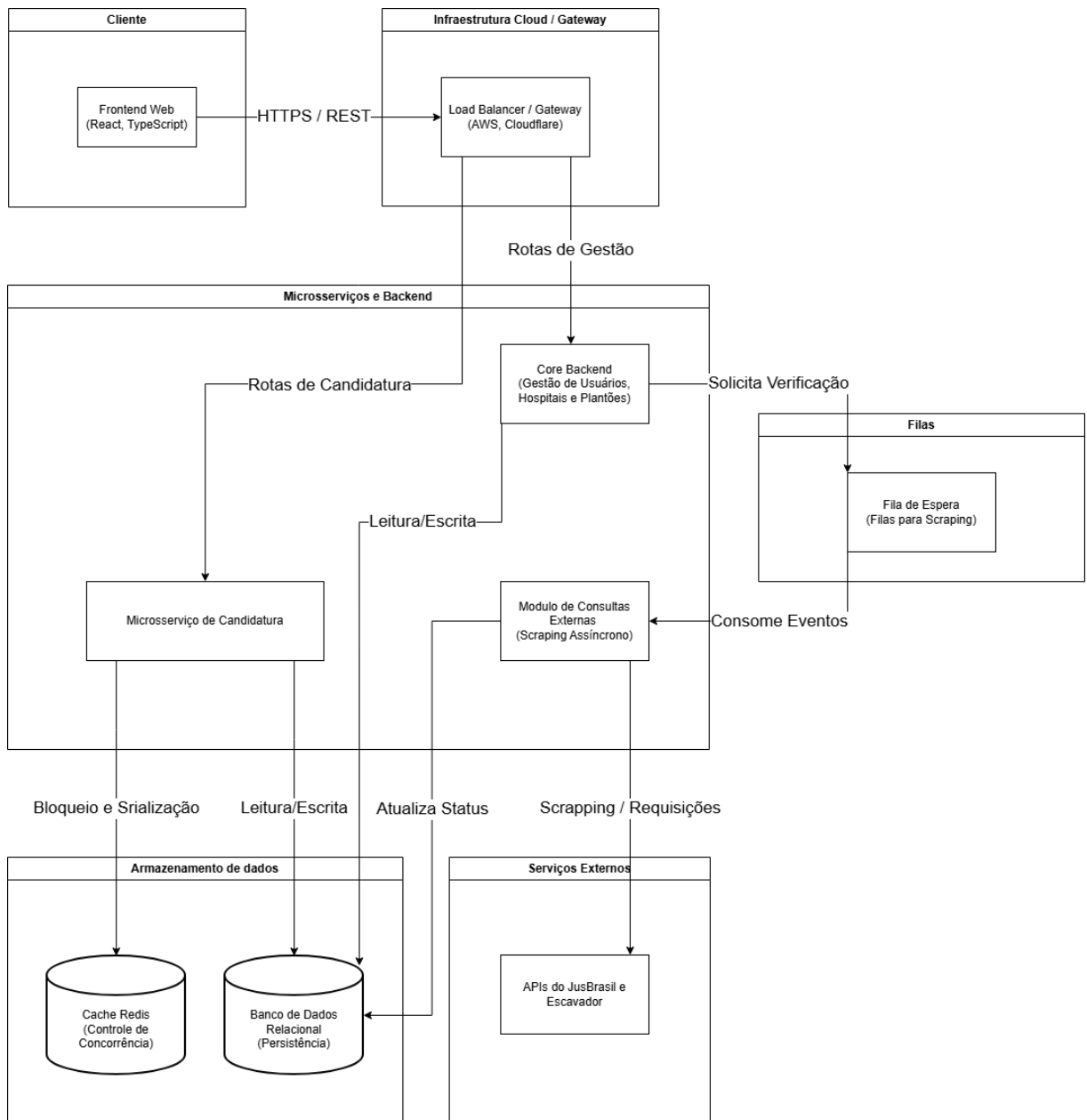


Figura 20

Padrões GoF

Singleton

Aplicação: Beans do Spring

Service e repositories

Strategy:

Aplicação: Diferentes estratégias de scraping e filtragem dos dados

Proxy:

Aplicação: API gateway atuando como intermediário entre o cliente e os serviços reais

Aplicação de SOLID

Princípios SOLID

SRP: Single responsibility principle

Exemplo:

- PlantaoController
- PlantaoService
- PlantaoRepository
- User/Medico

DIP: dependency inversion principle

- Uso da inversão de independência do string

CONCLUSÃO

A implementação final concretizou os principais elementos propostos durante a fase de concepção arquitetural.

A utilização de microsserviços, autenticação JWT, PostgreSQL e React permitiu construir uma solução modular, segura e preparada para evolução futura.

Funcionalidades inicialmente previstas, mas ainda não implementadas, permanecem como oportunidades de evolução para versões futuras da plataforma.